Automated Tutorial Video Generation Pipeline

Approach

The system takes a structured Markdown (`.md`) file as input. By standardizing the input format, we remove ambiguity and create a reliable and reusable workflow.

Flow

```
Markdown Script
        ↓
Script Parsing
        ↓
Audio Generation (TTS)
        ↓
VS Code Automation + Screen Recording
        ↓
Subtitle Generation
        ↓
Video Assembly
        ↓
FINAL.mp4
```

The architecture is modular so that the same components can later be exposed as MCP tools and orchestrated by Codex or other agents without changing the core implementation.

Task Division

Person 1 – Script Processing & Pipeline

Responsibilities

- Define Markdown format
- Parse input script
- Manage execution order
- Pipeline integration

Output

Structured data for downstream modules.

Person 2 – Audio & Subtitle Generation

Responsibilities

- Text-to-Speech generation
- Audio processing
- Subtitle generation

Outputs

- `voice.mp3`
- `subtitles.srt`

Person 3 – VS Code Automation & Recording

Responsibilities

- Open VS Code
- Create files and type code
- Execute commands
- Screen recording

Output

- `screen_recording.mp4`

Person 4 – Video Assembly

Responsibilities

- Synchronization
- Merge video, audio, and subtitles
- Export final video

Output

- `final.mp4`

Git and Integration

- Create separate branches for each module.
- Develop and test modules independently.
- Define clear input/output interfaces between modules.

- Use pull requests for merging into the main branch.
- Perform integration testing after combining all modules.
- Keep modules independent and loosely coupled to maintain future MCP compatibility.

Future Scope

Current System

```
Markdown Script
        ↓
Python Pipeline
        ↓
Final Video
```

Future Extension

```
Markdown Script
        ↓
MCP Tools
        ↓
Codex / Agent Orchestration
        ↓
Final Video
```

This allows the project to evolve into an agentic workflow without requiring major changes to the existing codebase.